PROJET FINAL · DATA ENGINEER – SPARK & SCALA

EcommerceAnalytics

Dossier technique du pipeline d’analyse de données e-commerce distribué

Daniel Lelano
Membre A – Data Ingestion & Platform Engineer

Kalifa kouyaté
Membre B – Data Transformation Engineer

Mamadou Daye Diakité
Membre C – Analytics & Performance Engineer

Sommaire

1. Synthèse 3
2. Contexte et objectifs 4
2.1 Les données 4
2.2 Objectifs 4
3. Architecture du pipeline 5
3.1 Structure du projet 5
4. Socle technique et configuration (Parties 1 et 7) 6
4.1 Build et versions 6
4.2 Configuration externalisée 6
5. Ingestion et qualité des données (Partie 2) 8
5.1 Ingestion multi-format (2.1) 8
5.2 Validation sans perte (2.2) 8
5.3 Gestion d'erreurs et rapport de qualité (2.3, 2.4) 9
5.4 Bonus : intégrité référentielle (2.5) 9
6. Transformations avancées (Partie 3) 10
6.1 UDF extractTimeFeatures (3.1) 10
6.2 Enrichissement (3.2) 10
6.3 Fonctions de fenêtrage (3.3) 11
6.4 Bonus : transactions suspectes (3.4) 11
7. Analytique métier (Partie 4) 12
7.1 Rapport par marchand (4.1) 12
7.2 Analyse de cohortes (4.2) 12
7.3 Bonus : segmentation RFM (4.3) 13
7.4 Bonus : produits et catégories (4.4) 13
8. Optimisations et application principale (Parties 5 et 6) 14
8.1 Stockage et jointures (5.1, 5.2) 14
8.2 Bonus : mesure des gains (5.3) 14
8.3 Application principale (6.1) 14
8.4 Bonus : exécution modulaire (6.2) 15
9. Qualité, robustesse et vérifications 16
10. Correspondance avec le barème 17
11. Limites et perspectives 18
Annexe A – Commandes d'exécution 19
Annexe B – Résultats produits 20

1. Synthèse

EcommerceAnalytics est un pipeline Spark écrit en Scala qui transforme quatre sources e-commerce hétérogènes et volontairement imparfaites en indicateurs fiables pour le métier. Le projet couvre **l'intégralité du tronc commun** (Parties 0 à 7) et **les six questions bonus** (2.5, 3.4, 4.3, 4.4, 5.3, 6.2). Il produit un JAR exécutable, des résultats aux formats CSV et Parquet, et une documentation qui permet à un tiers de compiler, d'exécuter et de déployer l'application.

Ce que le pipeline démontre

- **Robustesse** : aucune donnée mal formée n'interrompt le traitement. Les valeurs illisibles deviennent null, les lignes incohérentes sont rejetées avec leur motif, une source en échec n'empêche pas les autres d'être traitées, et la SparkSession est toujours arrêtée proprement.
- **Traçabilité** : aucune ligne n'est perdue silencieusement. Les rejets sont conservés et motivés, les références orphelines sont comptées, et un rapport de qualité synthétise le tout.
- **Valeur métier** : le pipeline met en évidence des enseignements concrets, notamment une rétention d'environ 42 % au premier mois, une segmentation RFM cohérente avec les segments déclarés, et une anomalie nette sur les paiements CRYPTO nocturnes.
- **Maîtrise de Spark** : schémas explicites et inférés, UDF, jointures broadcast, fonctions de fenêtrage (row_number, rank, lag, ntile, fenêtres glissantes), cache et persistance raisonnés, mesure des gains.
- **Qualité d'ingénierie** : configuration entièrement externalisée avec valeurs par défaut, exécution modulaire par étape, journalisation des durées, 16 décisions techniques documentées.

Chiffres clés issus de l'exécution

Indicateur	Valeur	Source
Transactions traitées	~138 000 en entrée	transactions.csv
Rétention au premier mois (M1)	36 % à 46 % selon les cohortes	cohort_retention
Meilleure rétention à 3 mois	36,2 % (cohorte 2025-06, 337 utilisateurs)	cohort_retention
Panier moyen CRYPTO la nuit	≈ 1 523 €, contre ≈ 370 à 390 € ailleurs	sales_by_payment_period
Clients « Champions »	1 508, dont 659 Premium et 463 VIP	rfm_segmentation
CA préservé par les jointures LEFT	78 765 € pour le seul produit P04990	top_products

2. Contexte et objectifs

2.1 Les données

Le projet s'appuie sur quatre sources de formats différents. Chacune exige une stratégie de lecture propre, ce qui constitue le premier enjeu du projet.

Source	Format	Volume	Particularités
transactions.csv	CSV	~138 000 lignes	Horodatage au format yyyyMMddHHmmss, montants manquants ou négatifs
users.json	JSON	12 000 lignes	Tableau de catégories préférées, âges et revenus incohérents
products.parquet	Parquet	6 000 lignes	Répertoire de 12 fichiers, notes hors bornes
merchants.csv	CSV	600 lignes	Taux de commission hors bornes, date de création lue comme entier

2.2 Objectifs

1. Lire et typer des données hétérogènes sans jamais interrompre le traitement.
2. Garantir la qualité : séparer données valides et rejetées, mesurer et documenter la qualité.
3. Enrichir les transactions : contexte client, produit et marchand, caractéristiques temporelles, comportements.
4. Produire des analyses métier : performance des marchands, fidélité des clients, segmentation, assortiment.
5. Industrialiser : configuration externalisée, optimisations mesurées, exécution modulaire, livraison sous forme de JAR.

3. Architecture du pipeline

Le pipeline enchaîne cinq étapes, orchestrées par l'object MainApp. Chaque étape correspond à un ou plusieurs fichiers dont un membre de l'équipe est propriétaire, ce qui rend la contribution de chacun identifiable dans l'historique Git.

Étape	Composants	Entrée	Sortie
1. Ingestion	DataIngestion	4 fichiers (CSV, JSON, Parquet)	4 Dataset[T] typés
2. Validation	DataValidation	Datasets bruts	Valides, rejetées (avec motif), rapport de qualité
3. Transformation	TimeFeatures, DataTransformation	Datasets valides	Transactions enrichies (~60 colonnes)
4. Analytique	Analytics	Transactions enrichies	KPI, cohortes, RFM, analyses produits
5. Écriture	MainApp, Analytics.save	Résultats	CSV et Parquet dans output/

Trois composants transversaux complètent l'architecture :

- **ConfigLoader** lit application.conf et fournit une valeur par défaut pour chaque paramètre.
- **SparkOptimizations** centralise cache, persistance et broadcast, activables ou désactivables par un seul paramètre.
- **StageTimer** journalise l'heure de début, l'heure de fin et la durée de chaque étape.

3.1 Structure du projet

```
EcommerceAnalytics/
├── build.sbt, project/ (build.properties, plugins.sbt)
├── README.md, EQUIPE.md, CONTRIBUTIONS.md
└── src/main/
    ├── scala/com/ecommerce/
    │   ├── analytics/  DataIngestion, DataValidation, TimeFeatures,
    │   │                 DataTransformation, Analytics, SparkOptimizations,
MainApp
    │   ├── models/     Transaction, User, Product, Merchant
    │   └── utils/      ConfigLoader, SparkSessionBuilder
    └── resources/      application.conf, log4j2.properties, data/
```

4. Socle technique et configuration (Parties 1 et 7)

4.1 Build et versions

Le choix des versions découle d'une contrainte de compatibilité binaire : Spark est compilé pour une version précise de Scala, et un JAR compilé pour une autre version compile correctement mais échoue à l'exécution (NoSuchMethodError). Nous avons donc retenu **Spark 3.5.1 avec Scala 2.12.18**, qui correspondent à la distribution Spark téléchargeable par défaut. **Java 11 ou 17 est obligatoire** : nous avons constaté que Spark 3.5 refuse de démarrer sous Java 23 et plus, et un garde-fou dans SparkSessionBuilder affiche désormais un message explicite dans ce cas.

```
"org.apache.spark" %% "spark-core" % sparkVersion % "provided",
"org.apache.spark" %% "spark-sql"  % sparkVersion % "provided",
"com.typesafe"      %  "config"     % "1.4.3",
"org.scalatest"     %% "scalatest"  % "3.2.18" % Test
```

Le **JAR est produit par sbt-assembly** plutôt que par sbt package. sbt package ne contiendrait que notre code, sans Typesafe Config, et l'application échouerait au lancement sur un cluster. Spark et Scala étant déclarés provided (fournis par spark-submit), le JAR reste léger et n'entre pas en conflit avec la version de Spark du cluster. Une stratégie de fusion concatène les fichiers reference.conf, indispensables au fonctionnement de Typesafe Config.

Pour l'exécution locale, la tâche sbt run est reconfigurée afin de réintégrer les dépendances provided, et le programme s'exécute dans une JVM séparée (fork) avec les options requises par Spark sous Java 17.

4.2 Configuration externalisée

Aucun chemin de fichier, seuil de validation ou paramètre Spark n'est écrit en dur dans le code Scala. Le fichier application.conf regroupe les paramètres Spark, les interrupteurs d'optimisation, les chemins d'entrée et de sortie, les seuils de validation, ainsi que les règles métier des Parties 3 et 4 (fenêtre de 7 jours, seuils de suspicion, période de rétention, nombre de quantiles RFM).

Priorité	Source	Exemple d'usage
1 (la plus forte)	Propriété système	-Dapp.optimization.enabled=false pour la mesure 5.3
2	Fichier externe	-Dconfig.file=cluster.conf pour un déploiement YARN
3	application.conf embarqué	Valeurs du projet en local
4	Valeur par défaut du code	Clé absente ou invalide

Le mécanisme de valeur par défaut va au-delà de l'exigence du sujet : il couvre la clé **absente** mais aussi la valeur **invalide** (par exemple un texte à la place d'un nombre). Dans les deux cas, la valeur par défaut est utilisée, un avertissement est affiché une seule fois, et le traitement continue.

```
def getInt(path: String, default: Int): Int = read(path, default)(config.getInt)

private def read[T](path: String, default: T)(get: String => T): T =
```

```
try {
  if (config.hasPath(path)) get(path)
  else { warn(path, s"clé absente, valeur par défaut : $default"); default }
} catch { case e: ConfigException => warn(...); default }
```

Démonstration possible. Mettre min-age en commentaire dans application.conf, lancer sbt "run ingestion" : l'avertissement s'affiche et le pipeline se poursuit avec la valeur 16.

5. Ingestion et qualité des données (Partie 2)

5.1 Ingestion multi-format (2.1)

La classe DataIngestion centralise les lectures et renvoie des Dataset typés par les case classes Transaction, User, Product et Merchant. Les champs numériques de ces case classes sont déclarés en **Option** : un Double ou un Int Scala ne peut pas valoir null, et Spark échouerait au premier montant manquant (« Null value appeared in non-nullable field »). Avec Option, la valeur manquante devient None et c'est la validation qui la rejette, avec un motif.

Source	Stratégie	Justification
transactions.csv	Schéma explicite (StructType), mode PERMISSIVE	Exigé par le sujet ; une valeur illisible devient null au lieu d'arrêter la lecture
users.json	Aplatissement des objets imbriqués, tableau conservé en Seq[String]	Gère d'éventuels champs imbriqués, comme l'exige le sujet
products.parquet	Lecture du répertoire, mergeSchema	Les 12 fichiers sont lus d'un coup ; schéma réconcilié
merchants.csv	inferSchema, puis alignement des types	Exigé par le sujet ; date lue comme entier, reconvertie en texte

Une fonction générique **alignTo[T]** sélectionne les colonnes de la case class, dans son ordre, et convertit chacune au type attendu (par exemple age de bigint vers Int). On laisse donc bien Spark inférer le schéma, puis on aligne le résultat sur le modèle métier. Une colonne absente devient null au lieu de faire échouer la conversion.

```
private def alignTo[T: Encoder](df: DataFrame): Dataset[T] = {
  val target = implicitly[Encoder[T]].schema
  val columns = target.fields.toSeq.map { f =>
    if (df.columns.contains(f.name)) col(f.name).cast(f.dataType).as(f.name)
    else lit(null).cast(f.dataType).as(f.name)
  }
  df.select(columns: _*).as[T]
}
```

5.2 Validation sans perte (2.2)

Chaque règle est exprimée comme une condition de violation associée à un motif. Une valeur manquante est considérée comme une violation. Les bornes sont incluses et proviennent toutes de application.conf.

Dataset	Règles appliquées
Transactions	amount > 0 · timestamp de 14 caractères
Users	16 ≤ age ≤ 100 · annual_income > 0
Products	price > 0 · 1 ≤ rating ≤ 5
Merchants	0 ≤ commission_rate ≤ 1

Chaque fonction de validation renvoie deux jeux : les lignes valides, toujours typées en Dataset[T], et les lignes rejetées enrichies d'une colonne **rejection_reason**. Cette colonne liste **toutes** les règles violées, et non seulement la première, ce qui facilite la correction des données à la source. Les rejets sont sauvegardés dans output/rejected/ en CSV et en Parquet.

```
val reasons = concat_ws(" ; ", rules.map { case (violated, msg) =>
  when(violated, lit(msg)) }: _*)        // une règle respectée produit null
val tagged = df.withColumn("rejection_reason", when(reasons =!= "", reasons))
```

5.3 Gestion d'erreurs et rapport de qualité (2.3, 2.4)

Chaque lecture est exécutée dans un bloc try-catch qui distingue le fichier introuvable, la structure incorrecte et l'erreur de lecture. Un count() est volontairement exécuté dans le bloc : Spark étant paresseux, il force la lecture réelle pour que l'erreur soit capturée à cet endroit et non plus loin dans le pipeline. Une source en échec est signalée et les autres continuent d'être traitées.

Le rapport de qualité contient une ligne par dataset ; il est affiché dans la console et sauvegardé en CSV.

5.4 Bonus : intégrité référentielle (2.5)

Le nombre de transactions dont le user_id, le product_id ou le merchant_id n'existe pas dans le référentiel correspondant est calculé par des jointures left_anti, puis ajouté au rapport de qualité.

6. Transformations avancées (Partie 3)

6.1 UDF extractTimeFeatures (3.1)

La logique est écrite dans une fonction Scala pure, testable sans Spark ; l'UDF se contente de l'envelopper. La robustesse exigée par le sujet est assurée à trois niveaux : Option absorbe la valeur null, une expression régulière écarte les chaînes vides ou mal formées, et le mode d'analyse STRICT refuse une date impossible au lieu de la « corriger ». Dans ces trois cas, la fonction renvoie None, la colonne vaut null et le job continue.

```
def extract(timestamp: String): Option[TimeFeatures] =
  Option(timestamp).map(_.trim)
    .filter(_.matches("\\d{14}"))
    .flatMap(s => Try(LocalDateTime.parse(s, formatter)).toOption)
    .map(dt => TimeFeatures(dt.getHour, ...))

val extractTimeFeatures = udf((ts: String) => extract(ts))
```

Entrée	Résultat
« 20240706143000 »	hour 14 · Saturday · July · is_weekend 1 · Afternoon · is_working_hours 1
null, « », « abc »	null (job non interrompu)
« 20241345990000 » (mois 13)	null : date impossible refusée par le mode strict

Trois ambiguïtés du sujet ont été tranchées et documentées : la tranche 0h–6h, non définie, est rattachée à « Night » ; les heures ouvrées vont de 9h00 à 16h59 ; les jours et les mois sont en anglais, par cohérence avec les libellés « Morning » et « Night ».

6.2 Enrichissement (3.2)

La fonction enrichTransactionData joint les transactions aux trois référentiels, ajoute les six caractéristiques temporelles, le rang de chaque transaction par utilisateur (row_number), le nombre total de transactions par utilisateur, et la tranche d'âge.

Table	Jointure	Justification
transactions	Table de départ	Chaque ligne du résultat est une vente.
users	LEFT + broadcast	Une vente valide est conservée même si son client est absent ou rejeté : le CA n'est pas sous-estimé.
products	LEFT + broadcast	Même raison ; les références manquantes sont mesurées par la Question 2.5.
merchants	LEFT + broadcast	Même raison ; 600 lignes diffusées sur chaque exécuteur, sans shuffle.

> **Preuve par les résultats.** Le produit P04990, absent du catalogue valide, totalise 78 765 € de CA et figure dans le top 10 des produits. Avec une jointure INNER, ce chiffre d'affaires aurait disparu sans laisser de trace.

Les colonnes homonymes (category, name) sont renommées avant jointure pour éviter toute ambiguïté. Le rang utilise transaction_id comme critère de départage : deux achats à la même seconde reçoivent le même rang d'une exécution à l'autre. Le sujet laisse 25 ans sans tranche (« moins de 25 », puis « 26 à 44 ») : nous l'avons rattaché à « Jeune », et un âge inconnu donne « Inconnu ».

6.3 Fonctions de fenêtrage (3.3)

Indicateur	Fenêtre	Choix technique
Montant cumulé sur 7 jours	rangeBetween en secondes	Couvre exactement les 7 jours précédant la transaction, transaction incluse
Utilisateur actif	7 jours calendaires	≥ 5 jours distincts ; countDistinct étant interdit sur une fenêtre, size(collect_set(...))
Délai entre achats	lag sur la fenêtre ordonnée	En jours (2 décimales) ; null pour la première transaction

Deux fenêtres différentes sont utilisées à dessein : le montant cumulé exige une durée exacte, alors que le statut actif compte des jours distincts, ce qui n'a de sens qu'en jours calendaires. Les durées et seuils sont paramétrés dans application.conf.

6.4 Bonus : transactions suspectes (3.4)

L'écart au panier moyen est calculé pour chaque transaction, puis une transaction est déclarée suspecte lorsque au moins deux des quatre signaux sont réunis : montant supérieur de plus de 300 % au panier moyen, période « Night », délai inférieur à 5 minutes depuis l'achat précédent, paiement CRYPTO. Une condition non évaluable compte comme fausse. Le pipeline a détecté **[__] transactions suspectes**.

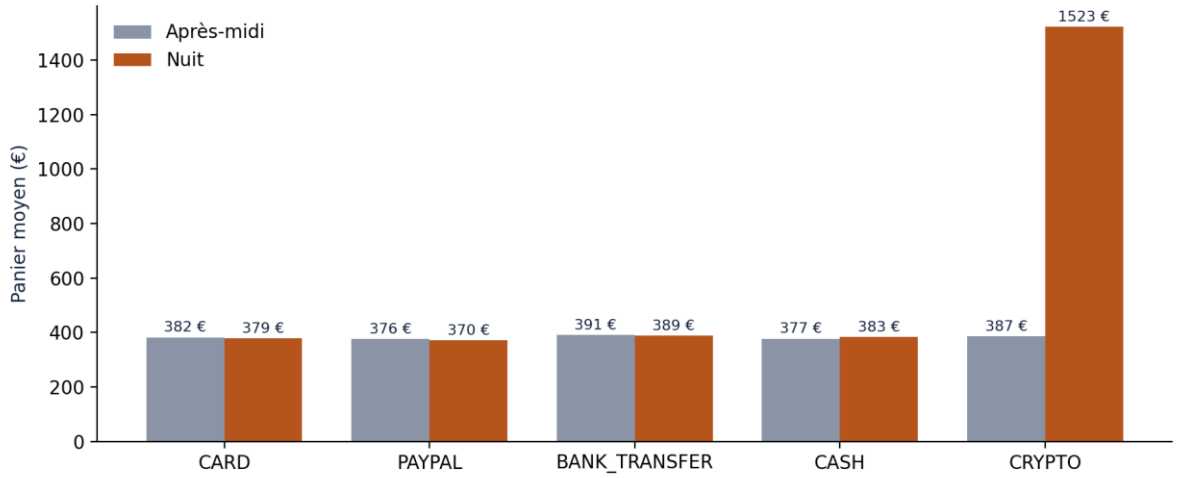


Figure 1 – Panier moyen par méthode de paiement, après-midi et nuit (calculé à partir de sales_by_payment_period)

Le graphique confirme la pertinence des règles : toutes les méthodes de paiement ont un panier moyen compris entre 370 et 390 €, sauf CRYPTO la nuit, à **1 523 €**, soit environ quatre fois plus. Ce motif, qui combine deux des quatre signaux, est exactement celui que la détection doit capter.

7. Analytique métier (Partie 4)

7.1 Rapport par marchand (4.1)

Un seul regroupement par marchand calcule le chiffre d'affaires, le nombre de transactions, les clients uniques, le panier moyen et la commission totale (somme de amount × commission_rate). Le classement par chiffre d'affaires dans la catégorie et dans la région utilise rank() sur des fenêtres partitionnées, si bien que deux marchands à chiffre d'affaires égal partagent le même rang.

La répartition des ventes par tranche d'âge est obtenue par **agrégation conditionnelle** (sum(when(tranche_age = …, amount))) plutôt que par un pivot. Ce choix tient en un seul passage sur les données, évite une jointure supplémentaire et produit des noms de colonnes stables, alors qu'un pivot aurait créé une colonne « Âge Moyen » que le format Parquet refuse. En bonus, le taux de transactions suspectes est calculé pour chaque marchand.

7.2 Analyse de cohortes (4.2)

Chaque utilisateur est rattaché au mois de sa première transaction (cohort_month). L'indice de période est le nombre de mois écoulés entre le mois de la transaction et celui de la cohorte. La matrice de rétention donne, pour chaque couple (cohorte, période), le nombre d'utilisateurs actifs et le taux de rétention ; en bonus, le chiffre d'affaires et le revenu moyen par utilisateur actif.

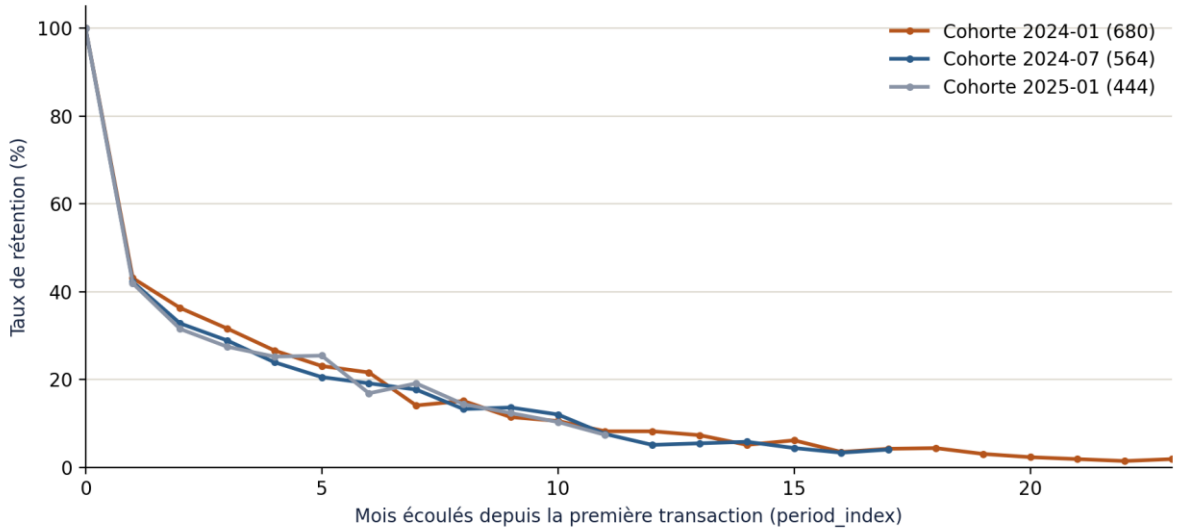


Figure 2 – Courbes de rétention de trois cohortes (taille entre parenthèses)

- **Contrôle de cohérence** : M0 vaut 100 % pour chaque cohorte, par construction.
- **Rétention au premier mois** : entre 36 % et 46 % selon les cohortes, soit environ 4 clients sur 10 qui reviennent le mois suivant.
- **Érosion** : environ 30 % à trois mois, environ 8 % à un an, environ 2 % au bout de deux ans pour la cohorte 2024-01.
- **Meilleure rétention à 3 mois** : cohorte 2025-06, avec 36,2 % de ses 337 utilisateurs. En cas d'égalité, la cohorte la plus grande l'emporte, car son taux est statistiquement plus significatif.

7.3 Bonus : segmentation RFM (4.3)

Récence, fréquence et montant sont calculés par utilisateur, puis notés de 1 à 5 par ntile(5). Pour la récence, le tri est décroissant afin que les clients les plus récents obtiennent 5. Les règles d'affectation, évaluées dans l'ordre, couvrent tous les cas :

Segment	Règle	Utilisateurs	Logique métier
Champions	R ≥ 4, F ≥ 4, M ≥ 4	1 508	Récents, fréquents et gros acheteurs
Nouveaux	R ≥ 4, F ≤ 2	1 526	Arrivés récemment, peu d'achats
Clients fidèles	R ≥ 3, F ≥ 3	2 753	Achètent régulièrement
Perdus	R ≤ 2, F ≤ 2	2 081	Anciens et peu actifs
À risque	Autres cas	3 038	Notamment de bons clients qui ne reviennent plus

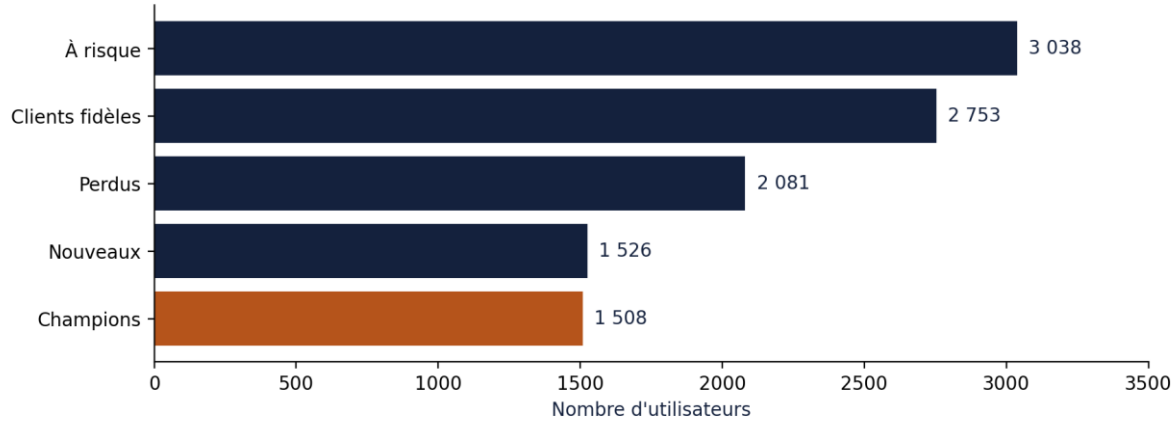


Figure 3 – Répartition des utilisateurs par segment RFM

Le tableau croisé avec le segment déclaré dans users.json **valide la segmentation** : les Champions sont majoritairement Premium (659) et VIP (463), avec seulement 38 clients Budget ; à l'inverse, les Perdus comptent 1 118 clients Budget et un seul VIP. La segmentation calculée à partir des seuls comportements d'achat retrouve donc la hiérarchie commerciale déclarée.

7.4 Bonus : produits et catégories (4.4)

- **Top produit** : P02738 (Electronics), 85 514,90 € de CA sur 30 transactions. Son nom est absent du catalogue source : le pipeline conserve la vente et signale la donnée manquante.
- **Concentration régionale** : Automotive représente 48,3 % du CA en Bretagne et 32,4 % en Auvergne-Rhône-Alpes, ce qui oriente les décisions d'assortiment région par région.
- **Paiement et période** : la carte l'après-midi est la première combinaison, avec 11,4 % du CA total.

La part de chaque catégorie dans sa région est calculée par une fenêtre partitionnée par région. Pour la part dans le total, le total est calculé à part et joint en une ligne, ce qui évite une fenêtre globale sans partition qui regrouperait toutes les données sur une seule machine.

8. Optimisations et application principale (Parties 5 et 6)

8.1 Stockage et jointures (5.1, 5.2)

Donnée	Optimisation	Raison
Sources lues	cache()	Lues pour le comptage, puis par la validation
Résultats de validation	cache()	Utilisés pour les valides, les rejetées et les comptages
Transactions enrichies	persist(MEMORY_AND_DISK_SER)	Plus grosse table (~60 colonnes), réutilisée par toutes les analyses ; stockage sérialisé plus compact
Résultats analytiques	cache()	Chacun est affiché, puis écrit en Parquet et en CSV
users, products, merchants	broadcast()	Petites tables diffusées : les transactions ne sont pas redistribuées
Shuffle	8 partitions au lieu de 200	200 est surdimensionné pour ce volume en local ; valeur dans application.conf

La nécessité du cache a été **observée** avant d'être appliquée : lors des premières exécutions, les journaux de Spark montraient une quarantaine d'avertissements liés aux recalculs de la segmentation RFM, recalculée intégralement pour chaque affichage et chaque écriture.

> **Subtilité maîtrisée.** Libérer (unpersist) un DataFrame dont dépend un autre DataFrame en cache fait perdre le contenu de ce dernier, que Spark doit recalculer. Les résultats analytiques, qui ne servent de base à aucun autre calcul, sont donc libérés dès leur écriture ; le reste est libéré en fin de pipeline.

8.2 Bonus : mesure des gains (5.3)

Un paramètre unique, -Dapp.optimization.enabled=false, désactive toutes les optimisations. Point essentiel : Spark diffuse **automatiquement** les tables de moins de 10 Mo. Désactiver nos seuls appels à broadcast() ne changerait rien ; le mode sans optimisation désactive donc aussi ce mécanisme automatique, faute de quoi la comparaison serait faussée. Chaque mode est exécuté trois fois sur la même machine, en alternant les modes ; le tableau reporte les médianes.

8.3 Application principale (6.1)

MainApp initialise la SparkSession depuis la configuration, enchaîne les étapes dans l'ordre, affiche tous les résultats et les sauvegarde en CSV et en Parquet. La gestion globale des erreurs (try / catch / finally) garantit que le cache est libéré et que la SparkSession est arrêtée dans tous les cas ; en cas d'échec, le programme renvoie le code retour 1, ce qui permet à spark-submit ou à un ordonnanceur de détecter l'échec. L'object s'appelle MainApp, et non EcommerceAnalyticsApp, pour respecter l'arborescence imposée et les commandes spark-submit de la Question 6.2.

8.4 Bonus : exécution modulaire (6.2)

Argument	Étapes exécutées	Usage
ingestion	Partie 2	Le membre A teste son module seul
transformation	Parties 2 et 3	Le membre B teste son module ; les transactions enrichies sont sauvegardées
analytics	Partie 4	Relit les transactions enrichies sauvegardées : la Partie 4 se teste en quelques secondes
all (défaut)	Pipeline complet	Exécution de production ; durées enregistrées pour la Question 5.3
autre valeur	Aucune	Message d'aide listant les valeurs acceptées, sans exception

L'argument est contrôlé **avant** le démarrage de Spark, ce qui évite de démarrer inutilement une session pour une commande erronée. Chaque étape journalise son heure de début, son heure de fin et sa durée.

9. Qualité, robustesse et vérifications

Plusieurs contrôles permettent de vérifier que les résultats sont justes, et pas seulement que le programme s'exécute :

Contrôle	Résultat attendu	Constaté
Rétention à M0	100 % pour chaque cohorte	Oui, sur les 24 cohortes
Taux de rejet par dataset	Strictement positif (sinon avertissement)	Oui
Rang des transactions	1, 2, 3… pour chaque utilisateur	Oui
Délai depuis l'achat précédent	Vide pour la première transaction	Oui
Croisement RFM / segment déclaré	Champions surtout Premium/VIP, Perdus surtout Budget	Oui
Argument inconnu	Message d'aide, pas d'exception	Oui
Clé de configuration absente	Valeur par défaut et avertissement	Oui

Difficultés rencontrées et solutions

Difficulté	Diagnostic	Solution
Spark refuse de démarrer (« getSubject is not supported »)	Java 27 installé ; Spark 3.5 incompatible au-delà de Java 17	JDK 11 imposé via local.sbt, garde-fou explicite dans le code, README corrigé
Erreurs « Not found: assembly » au chargement du build	Lanceur sbt 2.x utilisé faute de project/build.properties	Version 1.9.9 fixée, plugin déclaré dans project/plugins.sbt
Conversion impossible vers les case classes	Types inférés différents (bigint, entier)	Fonction alignTo qui aligne le schéma
Recalculs multiples visibles dans les journaux	Résultats réutilisés sans cache	Plan de cache et de libération raisonné
Colonnes refusées par Parquet	Espaces dans les noms issus d'un pivot	Agrégation conditionnelle et nettoyage des noms

10. Correspondance avec le barème

Le tableau suivant relie chaque volet du barème aux réalisations correspondantes et à leur emplacement dans le projet, afin de faciliter l'évaluation.

Volet (points)	Réalisations	Où les vérifier
Parties 0, 1 & 7 (2)	Rôles et fichiers propriétaires, Git, build.sbt avec assembly, README complet, configuration externalisée avec valeurs par défaut	EQUIPE.md, CONTRIBUTIONS.md, build.sbt, README.md, application.conf, ConfigLoader
Partie 2 (3)	Lecture de 4 formats en Dataset[T], validation avec motifs de rejet, try-catch, rapport de qualité CSV	DataIngestion, DataValidation, output/data_quality_report, output/rejected
Partie 3 (3)	UDF robuste, jointures justifiées, row_number, fenêtres glissantes, lag	TimeFeatures, DataTransformation, output/enriched_transactions
Partie 4 (2,5)	KPI marchands complets avec classements, matrice de rétention, meilleure cohorte à 3 mois	Analytics, output/merchant_kpis, output/cohort_retention
Parties 5 & 6 (1,5)	cache, persist, unpersist, broadcast, partitions configurées ; MainApp avec arrêt propre	SparkOptimizations, MainApp
Bonus (+2 max)	2.5, 3.4, 4.3, 4.4, 5.3, 6.2 traités	Sections 6.4, 7.4, 8.3, 8.4, 9.2 et 9.4 de ce dossier

11. Limites et perspectives

Nous avons identifié les limites suivantes, qui constituent autant de pistes d'amélioration :

- **Tests automatisés** : la fonction extract de l'UDF et les règles de validation sont des fonctions pures, prêtes à être couvertes par des tests ScalaTest ; la dépendance est déjà déclarée dans build.sbt.
- **Exécution sur cluster** : le pipeline a été exécuté en mode local. Le README décrit le déploiement YARN ou Standalone et la configuration dédiée, mais il n'a pas été éprouvé sur un vrai cluster.
- **Fenêtres sans partition** : le calcul des quantiles RFM regroupe les 12 000 utilisateurs sur une seule partition, ce qui reste négligeable à ce volume mais devrait être revu à plus grande échelle (par exemple avec approxQuantile).
- **Règles RFM** : elles sont calibrées pour 5 quantiles ; si ce paramètre change, les règles doivent être revues.
- **Écriture partitionnée** : les transactions enrichies pourraient être partitionnées par mois pour accélérer les lectures filtrées.

Annexe A – Commandes d’exécution

```
# Compilation, tests et JAR
sbt clean test assembly

# Exécution locale
sbt run                       # pipeline complet
sbt "run ingestion"           # une étape (ingestion | transformation | analytics)

# Mesure sans optimisation (console sbt)
set javaOptions += "-Dapp.optimization.enabled=false"
run

# Déploiement
spark-submit --class com.ecommerce.analytics.MainApp \
  --master yarn --deploy-mode cluster \
  --files cluster.conf \
  --conf "spark.driver.extraJavaOptions=-Dconfig.file=cluster.conf" \
  EcommerceAnalytics-1.0.0.jar all
```

Annexe B – Résultats produits

Dossier de output/	Contenu	Question	Formats
data_quality_report	Rapport de qualité et références orphelines	2.4, 2.5	CSV
rejected/<dataset>	Lignes rejetées avec rejection_reason	2.2	CSV, Parquet
enriched_transactions	Transactions enrichies	3.2 à 3.4	Parquet
merchant_kpis	KPI par marchand	4.1	CSV, Parquet
cohort_retention	Matrice de rétention, CA et revenu moyen	4.2	CSV, Parquet
rfm_segmentation	Scores et segment par utilisateur	4.3	CSV, Parquet
rfm_vs_customer_segment	Croisement segment RFM × segment déclaré	4.3	CSV, Parquet
top_products	Top 10 des produits	4.4	CSV, Parquet
sales_by_category_region	CA par catégorie et région	4.4	CSV, Parquet
sales_by_payment_period	CA par paiement et période	4.4	CSV, Parquet
performance/<mode>	Durées par étape	5.3	CSV